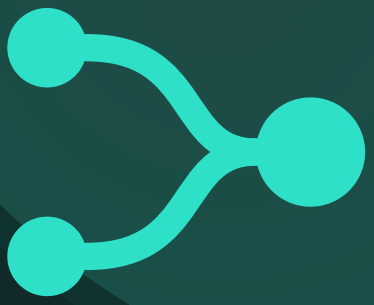
DevSync



Requirements Gathering

DevSync: Secure AI Project Collaborator

Stakeholders, stories, requirements and future ideas | Flutter graduation project

TEAM OF 5

Donia Alhosin
Bishay Talaat
Yousef Mohy
Emad Hany
Saga Maher

October 2026

Report map

This report covers the four required components of Requirements Gathering, plus a backlog of future requirements and ideas.

Required component	Where to find it
Stakeholder Analysis: key stakeholders and their needs	Part 1, sections 1 to 3
User Stories & Use Cases: scenarios of how users interact with the system	Part 2, sections 4 to 6
Functional Requirements: features and functionalities	Part 3, section 7
Non-functional Requirements: performance, security, usability and reliability	Part 4, section 8
Future requirements and ideas to make the project stronger; assumptions and open questions	Part 5, sections 9 and 10

Priority labels. Must = in the must-have scope, protected until 30 Nov. If time = built only if the core is done by the 12 Nov freeze. Future = after 30 Nov.

PART 1

Stakeholder Analysis

1. Stakeholders and their needs

Stakeholder	Role	Main needs	Influence	Interest
Developer / contributor	Primary user. Wants to join open-source projects.	Projects that fit their skills; a clear reason for each match; an easy way to ask to join; alerts when someone replies	Medium	High
Project maintainer	Primary user. Owns a project and needs contributors.	List a project simply; receive requests from relevant people; accept or decline quickly; see who the person is	Medium	High
Supervisor / lecturer	Assesses the project.	Clear plan, working demo, evidence of requirements, tests, security, documentation	High	High
Development team (5 members)	Builds and maintains DevSync.	Clear scope, fair workload, learning across all layers, a shippable result by 30 Nov	High	High
Pilot users	Early testers (target 20 or more).	Stable app, simple sign-in, relevant matches, a way to give feedback	Low	Medium
GitHub (API)	External service.	Respect rate limits and terms; read only public data	Medium	Low

Stakeholder	Role	Main needs	Influence	Interest
Firebase / Google Cloud	External service.	Stay within quota; correct security rules; billing alerts	High	Low
LLM provider	External service.	Usage within budget; safe handling of prompts and keys	Medium	Low
Open-source communities (future)	Wider community.	Healthy collaboration, no spam, respect for maintainers' time	Low	Medium

2. Influence and interest grid

The grid shows how much attention each stakeholder needs.

Keep satisfied (high influence, lower interest)
Firebase / Google Cloud

Manage closely (high influence, high interest)
Supervisor / lecturer, development team

Monitor (lower influence, lower interest)
GitHub API, LLM provider, open-source communities

Keep informed (lower influence, high interest)
Developers / contributors, maintainers, pilot users

3. How we gather and confirm requirements

Method	With whom	When	Output
Competitor review	Team	W1 (16-22 Oct)	Gaps and ideas (see the Literature Review)
Short interviews (5 developers, 2 maintainers)	Target users	W1-W2	Needs, pain points, wording for stories
Supervisor review of the plan and scope	Supervisor	W1, then weekly	Approved scope and KPIs
Prototype walkthrough (wireframes and first build)	Target users and team	W2-W3	Usability findings
Pilot with like/skip feedback and a 5-question survey	Pilot users	Late Nov	Match quality and satisfaction data
Backlog refinement in sprint planning	Team	Every Monday	Updated stories in Jira

PART 2

User Stories & Use Cases

4. Users

Persona (assumed)	Description	Goal
Nour, contributor	Final-year student who knows Flutter and Python and has a few small GitHub repositories	Find a real project to contribute to and build a portfolio
Karim, maintainer	Developer of a small open-source tool who needs help with testing and UI	Find two or three reliable contributors without posting everywhere

These personas are assumptions for design; the interviews in section 3 will confirm or change them.

5. User stories

ID	As a...	I want to...	So that...	Acceptance criteria	Priority
US-01	User	sign in with Google or email	I can use the app securely	Sign-in works with both providers; session persists after restart	Must
US-02	User	connect my GitHub account or username	the app knows my real activity	Languages, repositories and contributions are imported and shown	Must
US-03	User	add and edit my skills	matching reflects what I know	Skills can be added, removed and saved; changes affect the next match run	Must
US-04	Maintainer	list my project with link and needed skills	contributors can find it	A project with title, GitHub link and skills appears in the feed	Must
US-05	Contributor	see a ranked feed of projects with a score and reason	I know why each project fits	Feed shows score and a short reason; first results appear in 5 seconds or less	Must
US-06	Contributor	like or skip a match	the app learns what I prefer	Feedback is saved per user and project	Must
US-07	Contributor	send a collaboration request with a short message	the maintainer knows why I want to join	Request is stored as pending; duplicates are blocked	Must
US-08	Maintainer	get an alert when a request arrives	I can respond quickly	Push alert arrives within 10 seconds in 95% of tests	Must
US-09	Maintainer	accept or decline a request	I control who joins	Status updates for both users; requester gets an alert	Must

ID	As a...	I want to...	So that...	Acceptance criteria	Priority
US-10	User	see the status of my requests	I know what is pending	List shows pending, accepted and declined	Must
US-11	User	allow or deny notification permission when it is needed	I stay in control	Permission is asked at first use with an explanation; the app works if denied	Must
US-12	User	use smooth, clear animations	the app feels modern	Transitions on key screens; no jank on the test devices	Must
US-13	User	sign out and delete my account data	I control my privacy	Sign-out works; deletion removes profile and requests	Must
US-14	Team member	install Free or Pro build from a link	we can test and demo without a store	Install guide works on a clean device; the two flavors have different app IDs and icons	Must
US-15	Collaborator	use a shared task board for an accepted collaboration	we organize work together	Create, assign and move tasks; both members see updates	If time
US-16	Collaborator	get alerts about task changes	I do not miss updates	Alert on assignment and status change	If time
US-17	Team member	see a prompt-quality dashboard	we can compare prompt versions	Dashboard lists relevance per prompt version	If time

6. Use cases

Use case overview

ID	Use case	Contributor	Maintainer	External services	Stories
UC-01	Sign in	•	•	Firebase Auth	US-01
UC-02	Build profile from GitHub and skills	•	•	GitHub API	US-02, US-03
UC-03	List a project		•		US-04
UC-04	Get AI-ranked matches	•		LLM API, Firestore	US-05
UC-05	Like or skip a match	•		Firestore	US-06
UC-06	Send a collaboration request	•		Firestore, FCM	US-07
UC-07	Respond to a request		•	Firestore, FCM	US-08, US-09
UC-08	Track request status	•	•	Firestore	US-10
UC-09	Manage tasks for a collaboration (if time)	•	•	Firestore, FCM	US-15, US-16

Key use cases in detail

UC-01: Sign in

Actor	Contributor or maintainer
Precondition	App installed; internet available
Main flow	1. User opens the app and taps Sign in 2. User chooses Google or email 3. Firebase Auth verifies the user 4. App creates the user document on first sign-in and saves the FCM token 5. App opens the match feed or the profile setup
Alternative flows	2a. Wrong email or password: show a clear error and allow retry 2b. Google sign-in cancelled: return to the sign-in screen 3a. No internet: show an offline message
Postcondition	User is signed in and a user document exists

UC-04: Get AI-ranked matches

Actor	Contributor
Precondition	Signed in; profile has GitHub data or skills
Main flow	1. User opens the feed 2. App sends the profile and candidate projects to the matching function 3. The function returns a score and short reason for each project 4. App shows the ranked list with score meters
Alternative flows	3a. LLM is slow or fails: show skills-based ranking and a small notice 3b. No projects available: show an empty state with a hint 2a. Cached result is fresh: show it immediately
Postcondition	A ranked feed is shown; results are cached for the user

UC-06: Send a collaboration request

Actor	Contributor
Precondition	A match is open; user is signed in
Main flow	1. User taps Request to collaborate 2. User writes a short message and confirms 3. App stores a pending request 4. Cloud Function sends an FCM alert to the maintainer 5. User sees the request as pending
Alternative flows	3a. A request already exists: show its status instead 4a. Maintainer has no FCM token or notifications are off: request is still stored and shown in the app
Postcondition	Request is pending; maintainer is alerted

UC-07: Respond to a request

Actor	Maintainer
Precondition	A pending request exists
Main flow	1. Maintainer opens the alert or the requests list 2. Maintainer reads the profile, score and message 3. Maintainer accepts or declines 4. App updates the status 5. Cloud Function alerts the contributor
Alternative flows	3a. Maintainer ignores the request: it stays pending 5a. Contributor is offline: alert is delivered when the device reconnects
Postcondition	Status is updated for both users; contributor is notified

PART 3

Functional Requirements

7. Functional requirements

ID	Requirement	Stories	Priority
Authentication			
FR-01	The system shall let users sign in with Google using Firebase UI Auth.	US-01	Must
FR-02	The system shall let users register and sign in with email and password, and reset a password.	US-01	Must
FR-03	The system shall keep the session after restart and allow sign-out.	US-01, US-13	Must
FR-04	The system shall let a user delete their account and profile data.	US-13	Must
Profile and GitHub data			
FR-05	The system shall read public GitHub data for the user (languages, repositories, contributions).	US-02	Must
FR-06	The system shall let users add, edit and remove skills and a short bio.	US-03	Must
FR-07	The system shall cache GitHub data and refresh it in batches to respect rate limits.	US-02	Must
Projects			
FR-08	The system shall store projects with title, description, GitHub link and needed skills.	US-04	Must
FR-09	The system shall let a maintainer add and edit their own projects.	US-04	Must
FR-10	The system shall include seed projects so the feed is useful from day one.	US-05	Must
AI matching and feedback			
FR-11	The system shall build a structured profile and send it with a matching prompt to the model through a Cloud Function.	US-05	Must
FR-12	The system shall show a fit score and a short reason for each project.	US-05	Must
FR-13	The system shall rank projects by score and cache the result per user.	US-05	Must
FR-14	The system shall fall back to skills-based ranking if the model fails or times out.	US-05	Must
FR-15	The system shall let users like or skip a match and store the feedback.	US-06	Must
FR-16	The system shall keep versioned prompts and an evaluation set to compare prompt versions.	US-17	Must
FR-17	The system shall show a prompt-quality dashboard.	US-17	If time

ID	Requirement	Stories	Priority
Collaboration and notifications			
FR-18	The system shall let a user send one collaboration request per project with a short message.	US-07	Must
FR-19	The system shall let a maintainer accept or decline a request.	US-09	Must
FR-20	The system shall show request status (pending, accepted, declined) to both users.	US-10	Must
FR-21	The system shall send FCM alerts for new requests, status changes and project updates.	US-08, US-09	Must
FR-22	The system shall ask for notification permission at the moment of use with an explanation, and work if denied.	US-11	Must
Interface and animation			
FR-23	The system shall use implicit animations and advanced animation widgets (for example TweenAnimationBuilder and transitions) on key screens.	US-12	Must
FR-24	The system shall follow the DevSync design system (colors, fonts, spacing).	US-12	Must
Flavors and release			
FR-25	The system shall provide free and pro flavors with their own app ID, icon and Firebase configuration.	US-14	Must
FR-26	The system shall be built as obfuscated release builds and an Android App Bundle.	US-14	Must
FR-27	The team shall publish a signed build through an install link with a short install guide.	US-14	Must
Task management (if time allows)			
FR-28	The system shall let two collaborators create, assign and move tasks on a shared board.	US-15	If time
FR-29	The system shall send FCM alerts when a task is assigned or its status changes.	US-16	If time
FR-30	The system shall restrict the board to the two accepted collaborators with Firestore rules.	US-15	If time

Shaded rows are group headings. Each requirement is written to be testable, and the Test and Security report will link each FR to a test case.

PART 4

Non-functional Requirements

8. Non-functional requirements

ID	Category	Requirement	How it is verified
NFR-01	Performance	Match results are shown in 5 seconds or less for 95% of requests	Timing logs, Firebase Performance Monitoring
NFR-02	Performance	Cold start takes 3 seconds or less on the test devices	Manual timing, Performance Monitoring
NFR-03	Performance	An FCM alert is received within 10 seconds in 95% of tests	Test script with timestamps
NFR-04	Reliability	99% or more crash-free sessions	Firebase Crashlytics
NFR-05	Reliability	Backend services are 99.5% available during testing and the pilot	Firebase status and monitoring
NFR-06	Reliability	The app shows cached data and a clear message when offline, and falls back when the model fails	Manual offline tests, integration tests
NFR-07	Security	Firestore rules let users read and write only their own data and the requests they are part of	Rules unit tests and review
NFR-08	Security	No API keys or tokens in the repository; the LLM key is stored as a Cloud Function secret	Repository scan, review
NFR-09	Security	All traffic uses HTTPS; release builds are obfuscated and symbol files are stored safely	Network check, build review
NFR-10	Security	Runtime permissions are requested only when needed, with an explanation	Manual test, security checklist
NFR-11	Security	100% of the security checklist is completed before release	Checklist review
NFR-12	Privacy	Only public GitHub data and user-declared skills are used; users can delete their data	Data review, deletion test
NFR-13	Usability	Touch targets are at least 48 dp and text contrast meets WCAG AA (4.5:1)	Design review, contrast check
NFR-14	Usability	A new user can sign in and see matches in under 2 minutes	Walkthrough with pilot users
NFR-15	Usability	Errors are shown in plain language with a way to retry	Manual test
NFR-16	Maintainability	Every change is merged through a reviewed pull request; 70% or more coverage of matching logic	GitHub, CI
NFR-17	Maintainability	README and technical documentation are updated in the same pull request as the feature	Review checklist

ID	Category	Requirement	How it is verified
NFR-18	Compatibility	Runs on recent Android versions on at least two real devices and one emulator (minimum version to be confirmed)	Device test matrix
NFR-19	Scalability	Supports the pilot (20 or more users) within the free quota, with budget alerts	Firebase usage and alerts

PART 5

Future Requirements and Open Questions

9. Future requirements and ideas to make DevSync stronger

These ideas are outside the 30 Nov scope. Effort: S = under 1 day, M = 2-4 days, L = more than 4 days (planning estimates).

ID	Idea	Why it makes DevSync stronger	Effort
AI and matching			
FUT-01	Embedding pre-filter, then LLM ranking of top candidates	Lower cost and faster feed as the number of projects grows	M
FUT-02	Issue-level matching (good first issues)	Tells users which task to start with, not only which project	M
FUT-03	Skill-gap hints and a short learning path for each match	Turns a low score into a next step	M
FUT-04	Evidence in each reason (for example the repositories that support the score)	More trust in the AI explanation	S
FUT-05	Remote Config prompt versions with A/B tests, and automatic evaluation in CI	Improve prompts safely and measure the effect	M
Collaboration			
FUT-06	Maintainer search for contributors	Completes the two-sided marketplace	M
FUT-07	In-app chat after a request is accepted	Keeps the conversation in one place	L
FUT-08	Endorsements and badges after a collaboration	Builds reputation and encourages return visits	M
FUT-09	Team formation: suggest 3-4 people who fit one project together	Goes beyond one-to-one matching	L
Trust and safety			
FUT-10	Report and block users, rate limits on requests	Protects maintainers from spam	M
FUT-11	Firebase App Check and GitHub-verified profiles	Protects the backend and confirms identity	M
Product and platform			
FUT-12	Arabic localization with right-to-left layout	Reaches more local developers	M
FUT-13	Onboarding tour and shareable profile card	Higher profile completion and sharing	S
FUT-14	Web platform on the same Firebase backend	Reaches maintainers who work on desktop	L
FUT-15	iOS build and store release	Wider audience	L
FUT-16	Real Pro plan with in-app payments	Turns the Pro flavor into a business model	L

ID	Idea	Why it makes DevSync stronger	Effort
FUT-17	Analytics dashboard for match quality, adoption and requests	Helps the team decide what to build next	M

10. Assumptions and open questions

#	Assumption or question	Why it matters	Decide by
1	Projects in the feed come from seed data plus projects added by maintainers inside the app (FR-08 to FR-10).	The feed needs content before users arrive	W1
2	LLM calls run in a Cloud Function, which needs the Firebase Blaze plan (free quota applies).	Keeps the API key secret; also needed for FCM triggers	Before W3
3	The install link uses GitHub Releases or Firebase App Distribution with a signed, obfuscated build; the App Bundle is produced for the course requirement.	App Bundles cannot be installed directly on a phone	W4
4	Minimum supported Android version and the LLM provider and budget are to be confirmed.	Affects device testing and cost	W1
5	Stakeholder needs in section 1 are assumptions until the interviews in section 3 are done.	Stories may change	W2